

1-ma'ruza. Sinf e'loni. Obyekt yaratish. Konstruktorlar

Reja:

1. Sinfning umumiy shakli
2. Obyektlarni yaratish
3. Konstruktor va xossalardan foydalanish

Sinflar ushbu kitobning barcha namunaviy dasturlarida qo'llanilgan. Obyektga yo'naltirilgan dasturlashgacha foydalanilgan sinf eng oddiy sinf bo'lib, bu ularning ko'p xossalardan foydalanishga imkon bermaydi.

Sinflarning asosiy qoidalarini ko'rib chiqamiz. Sinf - bu obyekt shaklini belgilaydigan shablon. U ma'lumotlar va ushbu ma'lumotlarda ishlaydigan kodni belgilaydi. C# da sinfning nusxasi bo'lgan obyektlarni qurish uchun sinf spetsifikatsiyasidan foydalanadi. Sinf mantiqiy abstraksiya hisoblanadi. Sinfning fizik ko'rinishi operativ xotirada faqat shu sinf obyekti yaratilgandan keyingina paydo bo'ladi.

Sinfning umumiy shakli

Sinf aniqlanganda, uning tarkibidagi ma'lumotlar, shuningdek, ushbu ma'lumotlarda ishlaydigan kod e'lon qilinadi. Eng oddiy sinflar faqat kodni yoki faqat ma'lumotlarni o'z ichiga olishi mumkin bo'lsa-da, ko'pchilik haqiqiy sinflar ikkalasini ham o'z ichiga oladi.

Ma'lumotlar sinf tomonidan belgilangan a'zolarida, kod esa a'zo metodlarida mavjud. Sinflarning bir nechta a'zolari mavjud bo'lib, *metodlar*, *konstruktorlar*, *destruktorlar*, *indeksatorlar*, *hodisalar*, *operatorlar* va *xossalarni* o'z ichiga oladi.

`class` kalit so'zi bilan sinf yaratiladi. Quyida faqat o'zgaruvchilar nusxasi va metodlarini o'z ichiga olgan oddiy sinfning umumiy shakli keltirilgan.

```
class SinfNomi
{
    // O'zgaruvchilar e'loni
    ruxsat_modifikator tur_nomi uzgaruvchi1;
    ruxsat_modifikator tur_nomi uzgaruvchi2;
    //...
    ruxsat_modifikator tur_nomi uzgaruvchiN;
    // Metodlarni e'lon qilish.
    ruxsat_modifikator tur_nomi metod1(parametrlar)
    {
        // metod tanasi
    }
    ruxsat_modifikator tur_nomi metod2(parametrlar)
    {
        // metod tanasi
    }
    // ...
    ruxsat_modifikator tur_nomi metodN(parametrlar)
    {
        // metod tanasi
    }
}
```

Har bir o'zgaruvchi va metod e'lonidan oldin ruxsat modifikatori mavjud. Bu ma'lum bir sinf a'zosiga qanday kirishni ko'rsatuvchi ruxsat modifikatori hisoblanadi. Sinf a'zolari sinf ichida *private* yoki *public* bo'lishi mumkin. Ruxsat modifikatori ruxsat etilgan kirish turini belgilaydi. Ruxsat modifikatori ixtiyoriy, lekin yo'q bo'lsa, e'lon qilinayotgan a'zo sinf ichida *private* modifikatorini qabul qiladi. *private* a'zoldan faqat o'z sinfning boshqa a'zolari foydalanishi mumkin. Yuqoridagi dasturda `Main()` metodidan tashqari barcha a'zolar *public* sifatida

belgilangan. Bu shuni anglatadiki, ular boshqa barcha kod qismlarida, hatto sinfdan tashqarida aniqlanganlarda ham foydalanish mumkin.

Ruxsat modifikatori	Mazmuni, vazifasi
Public	Element sinfdan tashqarida va ichki sinflarda ishlatiladi.
Protected	Elementni sinfdan ishlatish mumkin emas, u faqat voris sinflarda ishlatiladi.
Private	Element faqat shu sinfdan ishlatiladi, uni boshqa sinflarda va voris sinflarda ishlatish mumkin emas.
Internal	Element faqat shu kompilyatsiya qilinuvchi dastur uchun ishlaydi, bu public va protected modifikatorlarining birlashmasidir.

Sinf a'zosi e'lonida ruxsat modifikatorlaridan tashqari bir yoki bir nechta modifikatorlar ham ishlatilishi mumkin.

Sinflarni ko'rsatish uchun biror bir binodagi qavatlar soni, umumiy maydoni va binoda yashovchi aholi soni haqidagi ma'lumotlarni saqlovchi Bino sinfini yaratishni ko'rib chiqamiz.

Quyida Bino sinfining birinchi varianti keltirilgan. U uchta o'zgaruvchisini belgilaydi: *Qavatlar*, *Maydoni* va *AholiSoni*. Bu yerda, Bino sinfinda metodlar umuman yo'q. Bu shuni anglatadiki, hozirgi vaqtda bu sinf faqat ma'lumotlardan iborat. (Keyinchalik metodlar ham unga kiritiladi)

```
class Bino
{
    public int Qavatlar; // Qavatlar soni
    public int Maydoni; // Binoning umumiy maydoni
    public int AholiSoni; // yashaydigan aholi soni
}
```

Bino sinfinda aniqlangan o'zgaruvchilar e'lon qilinadigan umumiy tartibni ko'rsatadi. Quyida sinf a'zosi hisoblangan o'zgaruvchilarni e'lon qilishning umumiy shakli keltirilgan:

`ruxsat_modifikatori tur_nomi o'zgaruvchi_nomi;`

Bu yerda, `ruxsat_modifikatori` kirish turini bildiradi; `tur_nomi` - o'zgaruvchining turi, `o'zgaruvchi_nomi` esa o'zgaruvchiga berilgan nom. Bino sinfinda e'lon qilingan barcha o'zgaruvchilar, *public* modifikatori bilan e'lon qilingan, shuning uchun bu o'zgaruvchilardan Bino sinfdan tashqarida ham foydalanishga imkon beradi.

Sinf ta'rifi yangi ma'lumotlar turini yaratishni bildiradi. Bunday holda, yangi ma'lumotlar turi Bino deb ataladi. Bino turidagi obyektlar shu nom bilan e'lon qilinishi mumkin. Ammo shuni unutmaslik kerakki, sinf nomi faqat turni tavsiflaydi, lekin aniq obyektни yaratmaydi. Shuning uchun yuqoridagi qism dasturda Bino turidagi obyektlar yaratilmagan.

Bino turidagi obyektни yaratish uchun quyidagicha kod yozish kerak bo'ladi.

```
Bino uy = new Bino(); // Bino turidagi obyekt yaratish
```

Kod bajarilgandan so'ng, uy obyektini Bino sinfining nusxasiga aylanadi.

Har safar sinfning nusxasi olinganda, ushbu sinfdan aniqlangan har bir o'zgaruvchining nusxasini o'z ichiga olgan obyektlar ham yaratiladi. Har bir Bino obyektini *Qavatlar*, *Maydonlar* va *AholiSoni* o'zgaruvchilarining o'z nusxalarini o'z ichiga oladi. Ushbu o'zgaruvchilarga kirish uchun *ObyektNomi* nuqta(.), *O'zgaruvchiNomi* orqali kiriladi. Nuqta(.) operatori obyekt nomini sinf a'zosi nomi bilan bog'laydi. Quyida nuqta operatorining umumiy shakli keltirilgan.

```
obyekt.azo;
```

Bunda obyekt chap tomonda, a'zo esa o'ng tomonda keltirilgan. Masalan, uy obyektining *Qavatlar* o'zgaruvchisiga 2 qiymatini belgilash quyidagicha bo'ladi.

```
uy.Qavatlar = 2;
```

Umuman olganda, nuqta operatori o'zgaruvchilar va metodlarga kirish uchun ishlatiladi. Quyida Bino sinfdan foydalanadigan dasturning to'liq dasturi keltirilgan.

```
using System;
class Bino
```

```

{
    public int Qavatlar; // Qavatlar soni
    public int Maydonlar; // Binoning umumiy maydoni
    public int AholiSoni; // Binoda yashaydigan aholi soni
}
// Bu sinf Bino turidagi obyektни e'lon qiladi.
class BinoDemo
{
    static void Main()
    {
        Bino uy = new Bino(); // Bino turidagi obyekt e'lon qilish
        int maydon; // kishi boshiga maydon
        //Uy obyektidagi maydonlarga qiymatlarni kiritish
        uy.AholiSoni= 4;
        uy.Maydonlar = 2500;
        uy.Qavatlar = 2;
        // Bir kishi uchun ajratilgan maydonni hisoblash.
        maydon = uy.Maydonlar / uy.AholiSoni;
        Console.WriteLine("Bino: \n " +
            uy.Qavatlar + " qavat\n " +
            uy.AholiSoni + " aholi\n " +
            uy.Maydonlar +
            " umumiy maydon bo'lib, shundan\n " +
            maydon + " kishi boshiga to'g'ri keladi!");
        Console.ReadKey();
    }
}

```

Bu dastur ikki sinfdan iborat: Bino va BinoDemo. BinoDemo sinfida Bino sinfining uy nusxasi avval Main() metodida yaratiladi, so'ngra ularga qiymatlar berish va keyin ushbu qiymatlardan foydalanish uchun uy obyektining o'zgaruvchilari Main() metodda ochiladi. Bino va BinoDemo alohida sinflardir. Ularning orasidagi yagona bog'liqlik shundaki, biri ikkinchisining nusxasini yaratadi. Lekin, bu alohida sinflar bo'lishiga qaramay, BinoDemo sinfidagi kod Bino sinfining a'zolariga kirish huquqiga ega, chunki ular *public* deb e'lon qilingan. Agar ular e'lon qilinganda ruxsat modifikatori ko'rsatilmagan bo'lsa, ularga kirish faqat Bino sinfida amalga oshiriladi va shuning uchun ularni BinoDemo sinfida ishlatib bo'lmaydi.

Yuqoridagi dastur kodi Bino.cs faylida saqlangan deb faraz qilaylik. Uning kompilyatsiyasi natijasida Bino.exe fayli yaratiladi. Bunday holda, Bino va BinoDemo sinflari avtomatik ravishda bajariladigan faylga kiritiladi. Ushbu dastur bajarilganda quyidagi natija ko'rsatiladi.

```

Bino:
2 qavat
4 aholi
2500 umumiy maydon bo'lib, shundan
625 kishi boshiga to'g'ri keladi!

```

Bino va BinoDemo sinflari bir xil faylda bo'lishi shart emas. Ularning har birini Bino.cs va BinoDemo.cs kabi alohida faylga joylashtirish va C# kompilyatoriga ikkala faylni birgalikda kompilyatsiya qilish kerakligini ko'rsatish mumkin.

Quyidagi asosiy prinsipni ko'rib chiqamiz: har bir obyekt o'z sinfida aniqlangan o'zgaruvchilarning nusxalariga ega. Ikkala obyekt o'rtasida hech qanday bog'liqlik yo'q. Demak, agar Bino turidagi ikkita obyekt bo'lsa, ularning har biri *Qavatlar*, *Maydonlar* va *AholiSoni* o'zgaruvchilarning nusxasiga ega bo'lib, ularning ikkala obyektidagi mazmuni ham farq qilishi mumkin.

```

// Ushbu dasturda Bino turidagi ikkita obyektни yaratiladi
using System;
class Bino
{
    public int Qavatlar; // Qavatlar soni
    public int Maydonlar; // Binoning umumiy maydoni
    public int AholiSoni; // Binoda yashaydigan aholi soni
}
// Bu sinf Bino turidagi obyektни e'lon qiladi
class BinoDemo
{
    static void Main()
    {
        Bino uy = new Bino(); // Bino turidagi obyekt e'lon qilish
        Bino tashkilot = new Bino();
        int maydon; // kishi boshiga maydon
            // Uy obyektidagi maydonlarga qiymatlarni kiritish
        uy.AholiSoni = 4;
        uy.Maydonlar = 2500;
        uy.Qavatlar = 2;
        // Tashkilot obyektidagi maydonlarga qiymatlarni kiritish
        tashkilot.AholiSoni = 25;
        tashkilot.Maydonlar = 4200;
        tashkilot.Qavatlar = 3;
        // Bir kishi uchun ajratilgan maydonni hisoblash
        maydon = uy.Maydonlar / uy.AholiSoni;
        Console.WriteLine("Bino: \n " +
            uy.Qavatlar + " qavat\n " +
            uy.AholiSoni + " aholi\n " +
            uy.Maydonlar +
            " umumiy maydon bo'lib, shundan\n " +
            maydon + " kishi boshiga to'g'ri keladi!");
            // Obyektdagi bir kishiga to'g'ri keladigan maydonni hisoblang.
        maydon = tashkilot.Maydonlar / tashkilot.AholiSoni;
        Console.WriteLine("Tashkilotda:\n " +
            tashkilot.Qavatlar + " qavat\n " +
            tashkilot.AholiSoni + " ishchilar\n " +
            tashkilot.Maydonlar +
            " umumiy maydon bo'lib, shundan\n " +
            maydon + " kishi boshiga to'g'ri keladi!");
        Console.ReadKey();
    }
}

```

Quyida ushbu dasturning natijasi keltirilgan.

Bino:

2 qavat

4 aholi

2500 umumiy maydon bo'lib, shundan

625 kishi boshiga to'g'ri keladi!

Tashkilotda:

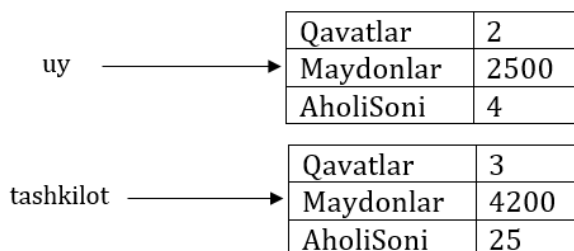
3 qavat

25 ishchilar

4200 umumiy maydon bo'lib, shundan

168 kishi boshiga to'g'ri keladi!

Bu yerda, uy obyektidagi ma'lumotlar tashkilot obyektidagi ma'lumotlardan butunlay ajralib turadi. Bu holat quyidagi rasmda aniq ko'rsatilgan.



Obyektlarni yaratish

Oldingi kod misollari Bino turidagi obyektни e'lon qilish uchun quyidagi kod qatoridan foydalangan.

```
Bino uy = new Bino();
```

Ushbu dastur kodi uchta funksiyani bajaradi. Birinchidan, uy o'zgaruvchisi e'lon qilinadi, u Bino sinf turiga tegishli. Bu o'zgaruvchining o'zi obyekt emas, faqat obyektga murojaat qila oladigan o'zgaruvchidir. Ikkinchidan, obyektning nusxasi yaratiladi. Bu *new* operator yordamida amalga oshiriladi. Uchinchidan, uy o'zgaruvchisi ushbu obyektga havola beriladi. E'lon qilingan uy o'zgaruvchisi Bino turidagi obyektga ishora qiladi.

new operator dinamik ravishda (ya'ni ish vaqtida) obyekt uchun xotira ajratadi va unga havolani qaytaradi, keyin esa o'zgaruvchida saqlanadi. Shuning uchun C# da barcha sinf obyektlari dinamik ravishda ajratilgan xotira bo'lishi kerak.

uy o'zgaruvchisining e'lonini u tegishli ob'yeht yaratilishidan quyidagicha ajratish mumkin.

```
Bino uy; // obyektga havolani e'lon qilish
```

```
uy = new Bino(); // Bino turidagi obyekt uchun xotira ajratish
```

Birinchi qator uy o'zgaruvchisini Bino turidagi obyektga havola sifatida e'lon qiladi. uy o'zgaruvchi bo'lib, u obyektning o'zi bo'lmasa ham, unga murojaat qilishi mumkin. Ikkinchi qatorda esa *new* Bino turidagi obyekt yaratiladi va unga havola uy o'zgaruvchisiga beriladi. Natijada, uy o'zgaruvchisi ushbu obyekt bilan bog'lanadi.

Sinf obyektlariga havola orqali kirish mumkinligi, sinf havola turlari deb atalishini anglatadi. Qiymat turi o'zgaruvchi ma'lum bir qiymatni o'z ichiga oladi. Masalan:

```
int x;
```

```
x = 10;
```

x o'zgaruvchisi 10 qiymatini o'z ichiga oladi, chunki u qiymat turi bo'lgan int turiga ega. Lekin quyidagi satr:

```
Bino uy = new Bino();
```

uy o'zgaruvchisi obyektning o'zini o'z ichiga olmaydi, faqat unga havola hisoblanadi.

Konstruktorlar

Yuqoridagi dasturlarda Bino turidagi har bir obyektning o'zgaruvchilari, xususan, quyidagi ketma-ketlikdan foydalangan holda qo'lda ishga tushirilishi kerak.

```
uy.AholiSoni = 4;
```

```
uy.Maydon = 2500;
```

```
uy.Qavatlar = 2;
```

Bu usul odatda C# kodida ishlatilmaydi. Shuningdek, u xatolik berishi ham mumkin (maydonlardan birini ishga tushirishni unutish mumkin). Biroq bu muammoni hal qilishning yaxshiroq yo'li konstruktordan foydalanish.

Konstruktor obyekt yaratilganda uni ishga tushiradi. Konstruktor o'z sinfi bilan bir xil nomga ega va sintaksis jihatidan metodga o'xshaydi. Ammo konstruktorlarda qiymat qaytarish turi yo'q. Quyida konstruktorning umumiy shakli keltirilgan.

```
Ruxsat modifikatori SinfNomi(parametrlar)
```

```

{
    // konstruktor tanasi
}

```

Odatda, konstruktor sinfda aniqlangan o'zgaruvchilarning boshlang'ich qiymatlarini o'rnatish yoki to'liq shakllangan obyektни yaratish uchun zarur bo'lgan boshqa funksiyalarni bajarish uchun ishlatiladi. Bundan tashqari, ruxsat modifikatori sinf bilan bir xil bo'ladi, chunki konstruktorlar odatda sinfda chaqiriladi. Parametrlar bo'sh yoki bir nechta parametrlardan iborat bo'lishi mumkin.

Barcha sinflarda konstruktorlar mavjud, C# avtomatik ravishda barcha o'zgaruvchilarni standart qiymatlarda ishga tushiradigan standart konstruktorni taqdim etadi. Ma'lumotlar turlari uchun standart qiymat null, bool false va havola turlar null bo'ladi. Ammo konstruktor aniqlangandan keyin, standart konstruktorlar ishlatilmaydi.

Quyida konstruktordan foydalanishga misol keltirilgan.

```

using System;
class MyClass
{
    public int x;
    public MyClass()
    {
        x = 10;
    }
}
class ConsDemo
{
    static void Main()
    {
        MyClass t1 = new MyClass();
        MyClass t2 = new MyClass();
        t2.x += 20;
        Console.WriteLine(t1.x + " " + t2.x);
        Console.ReadKey();
    }
}

```

Ushbu misolda MyClass sinfining konstruktori quyidagicha.

```

public MyClass()
{
    x = 10;
}

```

Parametrli konstruktorlar

Oldingi misolda parametri yo'q konstruktor ishlatildi. Ba'zi hollarda bu yetarli, ammo ko'pincha konstruktor bir yoki bir nechta parametrlarni qabul qilishi kerak. Parametrlar konstruktorga metodga o'xshash tarzda uzatiladi. Buning uchun ularni konstruktor nomidan keyin qavs ichida e'lon qilish kifoya. Quyida parametrli MyClass konstruktorga oid misol keltirilgan.

```

using System;
class MyClass
{
    public int x;
    public MyClass(int i)
    {
        x = i;
    }
}

```

```

class ParmConsDemo
{
    static void Main()
    {
        MyClass t1 = new MyClass(10);
        MyClass t2 = new MyClass(88);
        Console.WriteLine(t1.x + " " + t2.x);
    }
}

```

Destruktorlar

C# da keraksiz obyektlarni yo‘qotish imkoniyati mavjud bo‘lib, ular destruktur deb ataladi. Yo‘qotilgan obyekt tomonidan ajratilgan tizim resursi bo‘shatilganligini ta‘minlash uchun destruktordan foydalanish mumkin. Destruktorlar o‘ziga xos vositalar bo‘lib, kamdan kam hollarda qo‘llaniladi.

Quyida destruktorning umumiy shakli keltirilgan:

```

SinfNomi()
{
    // Destruktor tanasi
}

```

Bu yerda, SinfNomi - ma‘lum bir sinfning nomi. Destruktorlar ham konstruktor kabi e‘lon qilinadi, faqat “SinfNomi” dan oldin ~ belgisi qo‘yiladi. Shuni esda tutish lozimki, destruktorda qaytarish turi yo‘q va unga hech qanday argument berilmaydi.

Sinfga destruktur qo‘shish uchun uni sinf a‘zosi sifatida kiritish kerak. U o‘z sinfidagi obyektни o‘chirish kerak bo‘lganda chaqiriladi. Destruktorda obyektни yo‘q qilishdan oldin bajarilishi kerak bo‘lgan amallarni belgilash mumkin.

Ammo shuni yodda tutingki, destruktur obyektни yo‘q qilishdan oldin chaqiriladi. Masalan, obyektga havolani o‘z ichiga olgan o‘zgaruvchi ushbu obyekt doirasidan tashqarida bo‘lsa, u chaqirilmaydi. Bu destruktorni qachon chaqirish kerakligini oldindan bilib bo‘lmasligini bildiradi.

Quyida destruktordan foydalanishga oid dastur keltirilgan. Ushbu dastur ko‘p sonli obyektlarni yaratadi va yo‘q qiladi. Ushbu jarayonning bir nuqtasida "yo‘q qilish" faollashadi va keraksiz narsalarni yo‘q qilish uchun destrukturlar chaqiriladi.

```

// Destruktordan foydalanish
using System;
class Destruct
{
    public int x;
    public Destruct(int i)
    {
        x = i;
    }
    // Obyektни yo‘qotish uchun destruktur chaqiriladi
    ~Destruct()
    {
        Console.WriteLine("Yo‘q qilish: " + x);
    }
    // Obyektни yaratadi va uni darhol yo‘qotadi.
    public void Generator(int i)
    {
        Destruct o = new Destruct(i);
    }
}
class DestructDemo

```

```

{
    static void Main()
    {

        Destruct ob = new Destruct(0);

        for (int soni = 1; soni < 10000; soni++)
            ob.Generator(soni);
        Console.WriteLine("Tugadi!");
        Console.ReadKey();
    }
}

```

Bu dastur quyidagicha ishlaydi. Konstruktor x o'zgaruvchisini ma'lum qiymatga ishga tushiradi. Ushbu misolda x o'zgaruvchisi obyekt identifikatori sifatida xizmat qiladi va obyekt yo'qotilganda destruktur x qiymatini chiqaradi. Destruct turidagi obyekt yaratadigan va darhol yo'q qiladigan Generator() metodi chaqiriladi. Avval DestructDemo sinfida Destruct turidagi obyekt yaratiladi, so'ngra 100000 ta obyekt yaratiladi va birma-bir yo'q qilinadi.

Eslatma: WriteLine() metodi - Destruct() destrukturida faqat undan foydalanish uchun chaqiriladi. Odatda, destruktur faqat o'z sinfida aniqlangan o'zgaruvchilarga ta'sir qiladi.

Xossalar

Sinf a'zolarining yana bir turi xossadir. Xossalar nom, *get* va *set* kirish vositalaridan iborat. *get* va *set* kalit so'zlari o'zgaruvchining qiymatini olish va o'rnatish uchun ishlatiladi. Xossaning asosiy afzalligi shundaki, uning nomi ifodalar va qiymatlashlarda xuddi oddiy o'zgaruvchi nomi bilan bir xilda qo'llanilishi mumkin, lekin xossaga nom bo'yicha kirish avtomatik ravishda uning *get* va *set* qo'shimchalarini chaqiradi.

Quyida xossaning umumiy shakli keltirilgan:

```

Tur XossaNomi{
    get {
        // maydondan o'qish uchun kirish kodi
    }
    set {
        // maydonga yozish uchun kirish kodi
    }
}

```

Bu yerda, Tur xossaning turini bildiradi, XossaNomi esa xossaga beriladigan nomdir. Xossa aniqlangandan so'ng, xossaga nom bo'yicha kirish avtomatik ravishda mos keladigan kirishni chaqiradi. Bunga qo'shimcha ravishda to'plamga kirish vositasi xossaga beriladigan qiymatni o'z ichiga olgan value parametrini qabul qiladi.

Ammo shuni yodda tutish kerakki, xossalar maydonlarni saqlash uchun xotiradagi joyni aniqlamaydi, faqat maydonlarga kirishni nazorat qiladi. Bu shuni anglatadiki, xossaning o'zi maydonni ta'minlamaydi va shuning uchun maydon xossadan tashqarida belgilanishi kerak.

Quyida prop maydoniga kirish uchun MyProp xossasini belgilaydigan dasturning oddiy misoli keltirilgan. Ushbu misolda xossa faqat musbat qiymatlarni qabul qiladi.

```

using System;
class SimpProp
{
    int prop; // MyProp xossasi tomonidan boshqariladigan maydon
    public SimpProp() { prop = 0; }
    /* Bu xossa yopiq prop o'zgaruvchisiga kirish imkonini beradi.
       Bu faqat musbat qiymatlarni belgilashga imkon beradi. */
    public int MyProp
    {

```

```

        get
        {
            return prop;
        }
        set
        {
            if (value >= 0) prop = value;
        }
    }
}
// Xossadan foydalanish.
class PropertyDemo
{
    static void Main()
    {
        SimpProp ob = new SimpProp();
        Console.WriteLine("ob.MyProp ning boshlang'ich qiymati: " + ob.MyProp);
        ob.MyProp = 100; // qiymat belgilash
        Console.WriteLine("ob.MyProp ning joriy qiymati: " + ob.MyProp);
        // prop o'zgaruvchisiga manfiy qiymat berish mumkin emas
        Console.WriteLine("ob.MyProp xossasiga -10 qiymatini belgilashga urinish");
        ob.MyProp = -10;
        Console.WriteLine("ob.MyProp ning joriy qiymati: " + ob.MyProp);
        Console.ReadKey();
    }
}

```

Bu kod bitta yopiq prop maydonini va unga kirishni boshqaruvchi MyProp xossasini belgilaydi. Xossaning o'zi maydon xotiraning qayerida saqlanishini aniqlamaydi, u faqat maydonga kirishni nazorat qiladi. Bundan tashqari, prop maydoni yopiq, ya'ni unga faqat MyProp xossasi orqali kirish mumkin.

MyProp xossasi ochiq, ya'ni *public* sifatida ko'rsatilgan, unga sinfdan tashqarida ham kirish mumkin. Xossaning *get* bo'limida prop maydonidan qiymatni qaytaradi, *set* bo'limida esa, agar bu qiymat musbat bo'lsa, prop maydonining qiymatini o'rnatadi. MyProp xossasi prop maydonida saqlanishi mumkin bo'lgan qiymatlarni boshqaradi va bu xossaning asosiy vazifasi hisoblanadi.

MyProp xossasining turi - ham o'qish, ham yozish sifatida aniqlanadi, chunki u asosiy maydonga berilganlarni o'qish va yozish imkonini beradi. Biroq xossalar faqat o'qish/yozish uchun bo'lishi mumkin. Agar faqat o'qish uchun xossa yaratish kerak bo'lsa, unda bitta *get* bo'limini va agar faqat yozish xossasini yaratish kerak bo'lsa, unda bitta *set* bo'limini aniqlash kifoya.

this kalit so'zi

Metod chaqirilganda, chaqiruvchi obyektga havola avtomatik ravishda unga uzatiladi, ya'ni *this* metod chaqiriladigan obyekt. Bu havola *this* kalit so'zi bilan belgilanadi. Shuning uchun *this* kalit so'zi chaqirilgan metod amal qiladigan obyektни aniq bildiradi. Ushbu kalit so'zning yaxshiroq tushunish uchun, to'rtburchakning asosi va balandligi berilganda to'rtburchakning yuzasini qaytaradigan Yuza() metodini o'z ichiga olgan Rect sinfini yaratuvchi dastur misolini ko'rib chiqaylik.

```

using System;
class Rect
{
    public int A;
    public int H;
}

```

```

public Rect(int w, int h)
{
    A = w;
    H = h;
}
public int Yuza()
{
    return A * H;
}
}
class UseRect
{
    static void Main()
    {
        Rect r1 = new Rect(4, 5);
        Rect r2 = new Rect(7, 9);
        Console.WriteLine("r1 to'rtburchak: " + r1.Yuza());
        Console.WriteLine("r2 to'rtburchak: " + r2.Yuza());
        Console.ReadKey();
    }
}

```

Sinfning boshqa a'zolariga obyekt yoki sinf nomini qo'shmasdan to'g'ridan to'g'ri kirish mumkin. Shuning uchun operator

```
return A * H;
```

bo'lishi mumkin.

Yuza() metodida chaqiruvchi bilan bog'langan A va H o'zgaruvchilarning nusxalari ko'paytirilishini hamda metod ularning qiymatini qaytarishini anglatadi. Xuddi shu operatorni quyidagicha yozish mumkin.

```
return this.A * this.H;
```

Ushbu e'londa *this* kalit so'zi Yuza() metodi chaqiriladigan obyektни bildiradi. Shuning uchun *this.A* ifodasida ushbu obyektning A o'zgaruvchisi nusxasiga, *this.H* ifodasida esa xuddi shu obyektning H o'zgaruvchisi nusxasiga havola qilinadi. Shunday qilib, agar Yuza() metodi x obyektida chaqirilgan bo'lsa, yuqoridagi koddagi *this* kalit so'zi x obyektiga havolani bildiradi.

this kalit so'zidan konstruktorda ham foydalanish mumkin. Bunday holda, u yaratilayotgan obyektни bildiradi. Masalan, Rect() metodidagi quyidagi kodni

```
A = w;
```

```
H = h;
```

shunday yozish mumkin edi.

```
this.A = w;
```

```
this.H = h;
```

Quyida *this* havolasi yordamida yozilgan Rect sinfi keltirilgan.

```

using System;
class Rect
{
    public int A;
    public int H;
    public Rect(int w, int h)
    {
        this.A = w;
        this.H = h;
    }
    public int Yuza()

```

```
    {  
        return this.A * this.H;  
    }  
}  
class UseRect  
{  
    static void Main()  
    {  
        Rect r1 = new Rect(4, 5);  
        Rect r2 = new Rect(7, 9);  
        Console.WriteLine("r1 to 'rtburchak yuzasi: " + r1.Yuza());  
        Console.WriteLine("r2 to 'rtburchak yuzasi: " + r2.Yuza());  
        Console.ReadKey();  
    }  
}
```